

2015 Summer Retreat Report: A Walk with PolyApps

Polyhedral Optimization of Scientific Applications Using AlphaZ

Swetha Varadarajan, Colorado State University

Executive Summary

This report summarizes research conducted between 2014 and 2016 on compiler optimization using the Polyhedral Equational Model (PEM) and the AlphaZ design-space exploration framework. The research focused on expressing affine computations using the Alpha/Alphabets domain-specific language (DSL), applying polyhedral transformations, automatically generating optimized OpenMP C code, and evaluating the resulting implementations through functional verification and performance analysis.

Research Objectives

- Investigate compiler optimization using the Polyhedral Equational Model.
- Learn and utilize the AlphaZ framework for design-space exploration.
- Rewrite affine regions of scientific applications using the Alpha/Alphabets DSL.
- Apply affine scheduling, loop transformations, and locality optimizations.
- Generate optimized OpenMP C code automatically.
- Verify correctness of transformed implementations.
- Evaluate performance using profiling and benchmarking tools.
- Compare generated code against existing compiler optimization frameworks.

Scientific Applications Investigated

The affine portions of the following applications were rewritten and optimized using AlphaZ.

- McCaskill RNA Partition Function [2]
- Nussinov RNA Folding [3]
- NuPACK [4]
- UNAFold [5]
- BPPMax RNA Secondary Structure Prediction [6]
- piRNA [7]
- IRIS [8]
- ABC Quantum Reactive Scattering Program [9]

Microbenchmark Studies

CPU microbenchmarks were implemented to evaluate individual optimization techniques.

- Matrix Multiplication (MM)
- Triangular Matrix Inversion (TMI)
- Optimal String Parenthesization (OSP)
- DSYR2K
- Maximum Segment Sum (MSS)

GPU microbenchmarks focused on evaluating kernel optimization techniques for:

- Matrix Multiplication and optimized variants
- Knapsack

Research Methodology

For each application, the following workflow was performed.

- Analyze the original implementation.
- Identify affine loop nests suitable for polyhedral optimization.
- Rewrite computational kernels using the Alpha/Alphabets DSL.
- Apply affine scheduling and legality-preserving loop transformations.
- Generate optimized OpenMP C code using AlphaZ.
- Perform functional verification against the original implementation.
- Conduct performance evaluation using representative benchmark datasets.

Compiler Frameworks Evaluated

Generated implementations were compared with code produced using:

- AlphaZ [1]
- Intel C Compiler (ICC)
- GNU Compiler Collection (GCC)
- Pluto Automatic Polyhedral Optimizer [10]

Development and Performance Analysis Tools

Development and debugging were carried out using:

- Eclipse IDE
- NVIDIA Nsight
- Integer Set Library (ISL) [11]

Performance profiling and hardware analysis utilized:

- Intel VTune Amplifier
- Linux Perf
- Valgrind [12]
- PAPI Performance API [13]
- Intel Processor Counter Monitor (PCM)
- Intel Memory Latency Checker (MLC)

Benchmarking and memory analysis were performed using:

- Roofline Toolkit [14]
- STREAM Memory Bandwidth Benchmark [15]
- CacheBench
- LMBench
- MemBench

Major Outcomes

- Successfully implemented the polyhedral portions of multiple large scientific applications using AlphaZ.
- Applied affine scheduling, loop fusion, permutation, tiling, and parallelization techniques.
- Automatically generated optimized OpenMP C implementations.
- Verified functional correctness of generated code.
- Performed comparative compiler evaluations using AlphaZ, ICC, GCC, and Pluto.
- Profiled applications using modern hardware performance analysis tools.
- Developed expertise in compiler optimization, performance engineering, scientific computing, and high-performance computing.

References

- [1] Yuki, T., Gupta, G., Kim, D., Pathan, T., & Rajopadhye, S. *AlphaZ: A System for Design Space Exploration in the Polyhedral Model*. Lecture Notes in Computer Science, Vol. 7760, Springer, 2013.
- [2] McCaskill, J. S. *The Equilibrium Partition Function and Base Pair Binding Probabilities for RNA Secondary Structure*. Biopolymers, 1990.
- [3] Nussinov, R., Pieczenik, G., Griggs, J. R., & Kleitman, D. J. *Algorithms for Loop Matchings*. SIAM Journal on Applied Mathematics, 1978.
- [4] Zadeh, J. N., et al. *NUPACK: Analysis and Design of Nucleic Acid Systems*. Journal of Computational Chemistry, 2011.
- [5] Markham, N. R., & Zuker, M. *UNAFold: Software for Nucleic Acid Folding and Hybridization*. Methods in Molecular Biology, 2008.
- [6] Rennie, W., et al. *BPMax: RNA Secondary Structure Prediction Using Max-Plus Algebra*. Bioinformatics, 2011.
- [7] Chitsaz, H., Salari, R., Şahinalp, S. C., & Backofen, R. *A Partition Function Algorithm for Interacting Nucleic Acid Strands*. Bioinformatics, 2009.
- [8] Pervouchine, D. D. *IRIS: Intermolecular RNA Interaction Search*. Genome Informatics, 2004.
- [9] Skouteris, D., Castillo, J. F., & Manolopoulos, D. E. *ABC: A Quantum Reactive Scattering Program*. Computer Physics Communications, 133(1), 2000.
- [10] Bondhugula, U., Hartono, A., Ramanujam, J., & Sadayappan, P. *PLUTO: A Practical and Fully Automatic Polyhedral Program Optimization System*. Proceedings of PLDI, 2008.
- [11] Verdoolaege, S. *isl: An Integer Set Library for the Polyhedral Model*. International Congress on Mathematical Software, 2010.
- [12] Nethercote, N., & Seward, J. *Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation*. Proceedings of PLDI, 2007.
- [13] Browne, S., Dongarra, J., Garner, N., Ho, G., & Mucci, P. *A Portable Programming Interface for Performance Evaluation on Modern Processors (PAPI)*. International Journal of High Performance Computing Applications, 2000.
- [14] Williams, S., Waterman, A., & Patterson, D. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. Communications of the ACM, 2009.
- [15] McCalpin, J. D. *STREAM: Sustainable Memory Bandwidth in High Performance Computers*, 1995.